KING FAISAL UNIVERSITY

College of Computer Sciences and Information Technology (CCSIT)

Department of Computer Engineering

CE 313 Computer Organization and Architecture

Project Report

Name of Group members	ID	Section
Atekah Hussain Aljafar	222427138	81
Fatimah Nasser Albesher	222411640	81
Fatimah Hussain Alabdellah	222412119	81

Submitted to:

Engr. Amani Bayer.

Table of Contents

Problem Statement 3
Introduction 4
Project Implementation with output screenshots 5
Conclusion 11
References 12

Problem Statement

Problem Description:

1.) Write a program in Assembly language that reads the numbers from memory **(num1: dw 16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32)**
and perform following operations on them:

- Separate the even numbers and store the result (even numbers) in consecutive memory locations in an array (dw) named **'even'**.

- Find the **average** of numbers from **even** array and store in array name
  avg : dw 0

- Sort the numbers from given array **(num2: dw 0, 1, 2, 3, 4, 5, 6, 7,8,9,10)** in descending order . Store the result in consecutive memory locations in an array (dw) named **sort**.

There should be separate <u>subroutines</u> for each mentioned task like **even**, **average**, **sort**. Also implement stack concept within each subroutines.

Introduction

This project focuses on implementing basic data processing tasks using assembly language. The program processes set of numbers stored in memory, performing operations such as separating the even numbers, averaging, and sorting. The project also focuses on structured programming principles by leveraging subroutines for each specific task and implementing stack operations to manage subroutine calls and local data.

Project Implementation with output screenshots

Code:

```
[ORG 0x2000]


        jmp        start


num1:     dw    16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32

even:     dw    0, 0, 0, 0, 0, 0, 0, 0, 0

sumEven:   dw    0

avg:      dw    0

num2:     dw    0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

sort:     dw    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0




EvenNum:

  mov bx, num1

  mov bp, even

  mov cx,17

loopeven:

  mov ax,[bx]

  and ax,1

  jnz skip

  mov ax,[bx]

  mov [bp], ax

  add bp, 2

skip:

  add bx,2
```

```
  loop loopeven
ret



sum:
   mov ax, 0
   mov cx, 9
sum_loop:
   add ax, [bx]
   add bx, 2
   loop sum_loop
   ret

average:
   mov cx, 9
   mov bx, ax
   xor dx, dx
   div cx
   ret



DecOrder:
   push cx
   push bx

   mov di, sort

FindPosition:
```

```
    cmp di, sort + 20
    jge InsertValue

    mov dx, [di]
    cmp dx, ax
    jl  ShiftRight
    jmp InsertValue

ShiftRight:
    mov [di + 2], dx
    add di, 2
    jmp FindPosition

InsertValue:
    mov [di], ax
    pop bx
    pop cx
    ret

start:
    call  EvenNum

    mov bx, even
    call sum
    mov [sumEven], ax

    call average
    mov [avg], ax
```

```
    mov  bx, num2 + 20
    mov  ax, [bx]
    mov  [sort], ax

    sub  bx, 2
    mov  cx, 10

SortLoop:
    mov  ax, [bx]
    call DecOrder

    sub  bx, 2
    loop SortLoop

    mov  ax, 0x4c00
    int  0x21
```

Output:

even		
	hex	dec
[even]	Register Value AX 0010	16
[even + 2]	Register Value AX 0012	18
[even + 4]	Register Value AX 0014	20
[even + 6]	Register Value AX 0016	22
[even + 8]	Register Value AX 0018	24
[even + 10]	Register Value AX 001A	26
[even + 12]	Register Value AX 001C	28
[even + 14]	Register Value AX 001E	30
[even + 16]	Register Value AX 0020	32

sumEven		
	hex	dec
[sumEven]	Register Value AX 00D8	216

avg		
	hex	dec
[avg]	Register Value AX 0018	24

sort		
	hex	dec
[sort]	Register Value AX 000A	10
[sort + 2]	Register Value AX 0009	9
[sort + 4]	Register Value AX 0008	8
[sort + 6]	Register Value AX 0007	7
[sort + 8]	Register Value AX 0006	6

[sort + 10]	Register Value AX 0005	5
[sort + 12]	Register Value AX 0004	4
[sort + 14]	Register Value AX 0003	3
[sort + 16]	Register Value AX 0002	2
[sort + 18]	Register Value AX 0001	1
[sort + 20]	Register Value AX 0000	0

Conclusion

The Assembly language program effectively demonstrates structured data manipulation through three tasks: separating even numbers, calculating their average, and sorting an array in descending order. By using distinct subroutines for each operation, the program enhances modularity and readability. The implementation of stack management highlights efficient resource handling in Assembly language. Overall, this program serves as a valuable educational tool, illustrating key principles of low-level programming, including data manipulation and memory management, while reinforcing the importance of structured programming practices.

References

- *Microprocessor - 8086 Instruction Sets*. (n.d.). https://www.tutorialspoint.com/microprocessor/microprocessor_8086_instruction_sets.htm